

LegalEase AI

Project Team

Saadullah	22I-8795
Ali Khan	22I-2547
Muhammad Uzair Siddique	22I-6181

Session 2022-2026

Supervised by

Ms.Fatima Gillani

Co-Supervised by

Mr.Farrukh Bashir



Department of Software Engineering

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

May, 2026

Contents

1	Introduction	1
1.1	Existing Solutions	1
1.2	Problem Statement	2
1.3	Scope	3
1.4	Modules	3
1.4.1	Case Management Module	3
1.4.2	AI Legal Chat Assistant	4
1.4.3	Document Analysis Module	4
1.4.4	AI Legal Research Module	4
1.4.5	Practice Simulator Module	5
1.4.6	Contract Drafting and Compliance Module	5
1.4.7	OCR and Scanned-Document Module	5
1.4.8	Security, Role Management, and Notifications Module	5
1.4.9	Unified Dashboard and User Interface Module	6
1.5	Work Division	6
2	Project Requirements	7
2.1	Use-case Diagram	7
2.2	Detailed Use Cases	9
2.2.1	UC-01: Login	9
2.2.2	UC-02: Create New Case	10
2.2.3	UC-03: Assign Client to Case	11
2.2.4	UC-04: Ask Legal Question via AI Chat	12
2.2.5	UC-05: Upload and Analyse Legal Document	13
2.2.6	UC-06: Search Legal Library by Semantic Query	14
2.3	Functional Requirements	14
2.3.1	2.2.1 Case Management Module	15
2.3.2	AI Legal Chat Assistant Module	15
2.3.3	Document Analysis Module	16
2.3.4	AI Legal Research Module	16
2.3.5	Contract Drafting and Compliance Module	17
2.3.6	OCR and Scanned-Document Module	17

2.3.7	Practice Simulator Module	18
2.3.8	Notifications Module	18
2.4	Non-Functional Requirements	18
2.4.1	Reliability	18
2.4.2	Usability	19
2.4.3	Performance	19
2.4.4	Security	20
3	System Overview	23
3.1	Architectural Design	23
3.2	Data Design	27
3.3	Domain Model	29
3.4	Design Models	31
4	Implementation and Testing	41
4.1	Algorithm Design	41
4.2	External APIs/SDKs	47
4.3	Testing Details	47
	References	52
A	Appendices	53
A.1	Appendix A	53
A.1.1	Use Case Diagram example (Online Shopping System)	53
A.1.2	Detail Use Case Example	54
A.1.3	Event-Response Table for a Highway Intersection	55
A.1.4	Story Board Example For Android App	56
A.2	Appendix B	57
A.2.1	Domain Model Example For Online Shopping	57
A.3	Appendix C	58
A.3.1	Box And Line Example For Online Shopping	58
A.3.2	Architecture Pattern Example For Online Shopping	59
A.4	Appendix D	60
A.4.1	Activity Diagram	60
A.4.2	Class Diagram	61
A.4.3	Sequence Diagram	62
A.4.4	State Transition Diagram	62
A.4.5	Data Flow Diagram	63

List of Figures

2.1	Use Case Diagram	8
3.1	Box and Line Diagram	24
3.2	System Architecture Diagram	26
3.3	ERD Diagram	28
3.4	Domain model Diagram	30
3.5	Activity Diagram	32
3.6	Class Diagram	34
3.7	SD User Login Diagram	36
3.8	SD-2 AI Chat with Rag	38
3.9	SD Upload Document with OCR	40
4.1	Login and Issue JWT Pair	42
4.2	Validated Case Status Transition	43
4.3	Stream-and-Commit Document Upload	44
4.4	Build the Legal Library Index	45
4.5	RAG Answer Generation	46
A.1	Use Case Diagram for the Online Shopping System	53
A.2	Detail Use Case Example	54
A.3	Example of Event Response Table	55
A.4	Example of Story Board	56
A.5	Domain Model Example For Online Shopping Application	57
A.6	Box and Line Diagram For Online Shopping Application	58
A.7	Architecture Pattern For Online Shopping Application	59
A.8	Activity Diagram For Online Shopping Application	60
A.9	Class Diagram For Online Shopping Application	61
A.10	Sequence Diagram For Online Shopping Application	62
A.11	State Transition Diagram For Online Shopping Application	62

List of Tables

1.1	Comparison of Existing Solutions	2
1.2	Work Division by Team Member	6
2.1	Use Case UC-01: Login	9
2.2	Use Case UC-02: Create New Case	10
2.3	Use Case UC-03: Assign Client to Case	11
2.4	Use Case UC-04: Ask Legal Question via AI Chat	12
2.5	Use Case UC-05: Upload and Analyse Legal Document	13
2.6	Use Case UC-06: Search Legal Library by Semantic Query	14
4.1	External APIs / SDKs Used in FYP-1	47
4.2	Authentication Service Unit Tests	48
4.3	Case Management Service Unit Tests	48
4.4	Document Service Unit Tests	49
4.5	AI Legal Chat Service Unit Tests	49

Chapter 1

Introduction

The legal sector in Pakistan still works on paper for most day-to-day tasks. Lawyers handle many cases at once, store files in folders, and track hearing dates by hand. Clients have to call or visit their lawyer to get a simple update. Law students do not have a place where they can practice handling cases before they enter real courtrooms. These problems lead to missed deadlines, lost documents, and a slow learning curve for new lawyers. LegalEase AI is a web platform that solves these problems. It puts case management, document handling, AI-based legal help, and a practice space for students into one web site. Lawyers can manage cases and clients. Clients can see their case progress online. Students can practice on simulated cases. The platform also has an AI chat assistant that answers legal questions in English and Urdu, using a stored library of Pakistani statutes and court judgments. This chapter gives the background of the project, lists the existing solutions, states the problem, defines the scope, lists the modules, and shows how the work is divided among the team members.

1.1 Existing Solutions

A few products already work in this space, both in Pakistan and elsewhere. None of them cover the full set of features that lawyers, clients, and students need at the same time

Table 1.1: Comparison of Existing Solutions

System Name	System Overview	System Limitations
Adalat Online [1]	A Pakistani portal that connects clients with lawyers and lists basic case information.	No AI features, no document handling, no role-based dashboards, no support for students.
Lexiber AI [10]	Only research; no case management, no OCR, no contract drafting, no Urdu support	Only research; no case management, no OCR, no contract drafting, no Urdu support
Pakistan Law Bot [14]	A simple chatbot trained on a few Pakistani laws.	Just a chatbot. No document upload, no case tracking, no role-based access. Answers do not cite sources.
LawScript [8]	An AI legal research website with template generation.	Only research and templates; no client portal and no audit logs.
YourMunshi [21]	An AI legal Q&A tool for Pakistani law in English	No Urdu support, no document handling, no support for students.

These existing tools are narrow. Each one does one thing only. LegalEase AI is different because it joins case management, AI document help, contract checks, and a learning space for students into one role-based platform

1.2 Problem Statement

Legal work in Pakistan still depends on manual record keeping and scattered communication. A lawyer in active practice can have thirty to fifty cases at the same time, with hearings, client messages, scanned filings, and notes for each one. Case data is spread across paper files, mobile messaging apps, and personal email. This makes it easy to miss deadlines and lose track of document versions. Clients have no proper channel to check the progress of their own case, so they have to call or visit the lawyer for every small update. Legal research is also slow. Pakistani case law is published on different court websites, and there is no single search tool that can look across statutes and judgments. Junior lawyers and law students spend a lot of time looking for the right precedent. Students also do not have a place where they can practice drafting petitions or applying a law to a problem before they face the same situation in a real courtroom. LegalEase AI solves

these problems by giving each user role its own workspace inside one website. Lawyers get a structured case management area. Clients get a read-only portal where they can see updates. Students get an AI assistant and a practice simulator. The AI features cite the source statute or judgment for every answer, so users can trust the output. With everything in one place, lawyers miss fewer deadlines, clients get clear updates, and students get hands-on practice

1.3 Scope

The scope is bounded so that the team can finish the work within FYP-1 and FYP-2.

In scope: A responsive web platform with three user roles (Lawyer, Client, Student/Intern). Lawyers can create and manage cases, assign clients, attach documents, schedule hearings, and view a timeline of case events. Clients can view their own cases and shared documents. Students can use the AI chat assistant and the practice simulator. The AI chat supports English and Urdu and uses a library of Pakistani statutes and judgments. Each answer cites the source. The system supports OCR of scanned PDFs and images, summarisation of long documents, semantic search across the legal library, and template based contract drafting with simple compliance checks. All access is controlled by JWT authentication and role-based permissions. Every important action is written to an audit log.

Out of scope: The platform does not give professional legal advice and is not a replacement for a licensed lawyer. It does not file petitions in court and does not produce certified court documents. Live courtroom transcription, voice input, online payments, and integration with private law-firm accounting systems are not included. Outcome prediction and a native mobile app (iOS / Android) are left for after FYP-2

1.4 Modules

The system is split into nine modules. Each module owns a clear part of the work, and all modules talk to each other through one FastAPI backend

1.4.1 Case Management Module

This module manages the full life of a legal case. It links cases to participants, hearings, deadlines, and documents

1. Create, update, archive, and close case records.

2. Assign clients and co-counsel with role-specific permissions.
3. Upload and organise case-related documents.
4. Schedule hearings and deadlines with automatic reminders
5. Show a timeline of every event on a case

1.4.2 AI Legal Chat Assistant

A chat tool that answers legal questions in plain language and cites the source.

1. Free-text legal Q&A in English and Urdu.
2. Search a library of Pakistani statutes and judgments to find supporting passages.
3. Show citations next to every answer.
4. Save chat history for each user and case

1.4.3 Document Analysis Module

Reads uploaded legal documents and pulls out the important parts

1. Open and read PDF or text documents.
2. Make a short summary of long judgments and contracts
3. Mark key clauses, parties, and dates.
4. Flag missing or risky clauses in contracts

1.4.4 AI Legal Research Module

A search tool over the legal library

1. Search statutes and judgments by typing a normal-language query.
2. Rank results by meaning, not just keywords.
3. Filter results by court, year, and case type.
4. Show similar past cases for any selected judgment

1.4.5 Practice Simulator Module

A learning space for law students and interns

1. Library of practice exercises (drafting, cross-examination, applying a law).
2. Interactive chat with a virtual client or judge.
3. Automatic feedback on the student's answers.
4. Track each student's progress over time

1.4.6 Contract Drafting and Compliance Module

Helps users write and check contracts.

1. Draft contracts from saved templates.
2. Suggest clauses while drafting.
3. Check the contract against compliance rules.
4. Compare versions and keep a version history

1.4.7 OCR and Scanned-Document Module

Turns scanned PDFs or images into editable text.

1. OCR for PDF and image files.
2. Extract text from scanned FIRs, contracts, and judgments
3. Pass the extracted text to the Document Analysis module.
4. Keep both the original scan and the text version

1.4.8 Security, Role Management, and Notifications Module

Handles login, permissions, and alerts.

1. JWT-based login.
2. Role-based access for Lawyer, Client, and Student.

3. Encrypted storage of passwords and sensitive files.
4. Audit log for every important action.
5. In-app and email notifications for deadlines and updates

1.4.9 Unified Dashboard and User Interface Module

The web pages that each user role sees.

1. Separate dashboards for Lawyer, Client, and Student.
2. View of recent activity, alerts, and pending tasks.
3. One-click access to cases, documents, and AI tools.
4. Works on desktops, tablets, and mobile browsers

1.5 Work Division

The project is built by a three-member team. Each member owns a vertical slice of the system from the user interface down to the database, with shared work on testing and deployment. This direct response to the mid-evaluation comment about task allocation.

Table 1.2: Work Division by Team Member

Member (Roll No.)	Lead Role	Modules Owned
Ali Mehmood Khan (22I-2547)	Backend / AI	AI Legal Chat Assistant (RAG pipeline), AI Legal Research, Document Analysis, Practice Simulator.
Saadullah (22I-8795)	Frontend / UX	Unified Dashboard, role-based UI, Case Management UI, Notifications front-end, Contract Drafting UI.
Uzair Siddique (22I-6181)	Platform / Data	Authentication, RBAC, Case Management API, PostgreSQL schema, OCR pipeline, audit logging, deployment.
Shared		API contracts, requirements, testing, demo prep.

Chapter 2

Project Requirements

This chapter presents the use case diagram, functional requirements, non-functional requirements, and domain model for LegalEase AI. Each functional requirement carries an acceptance criterion. Each non-functional requirement has a measurable target and a verification method so that every requirement in this document can be tested.

2.1 Use-case Diagram

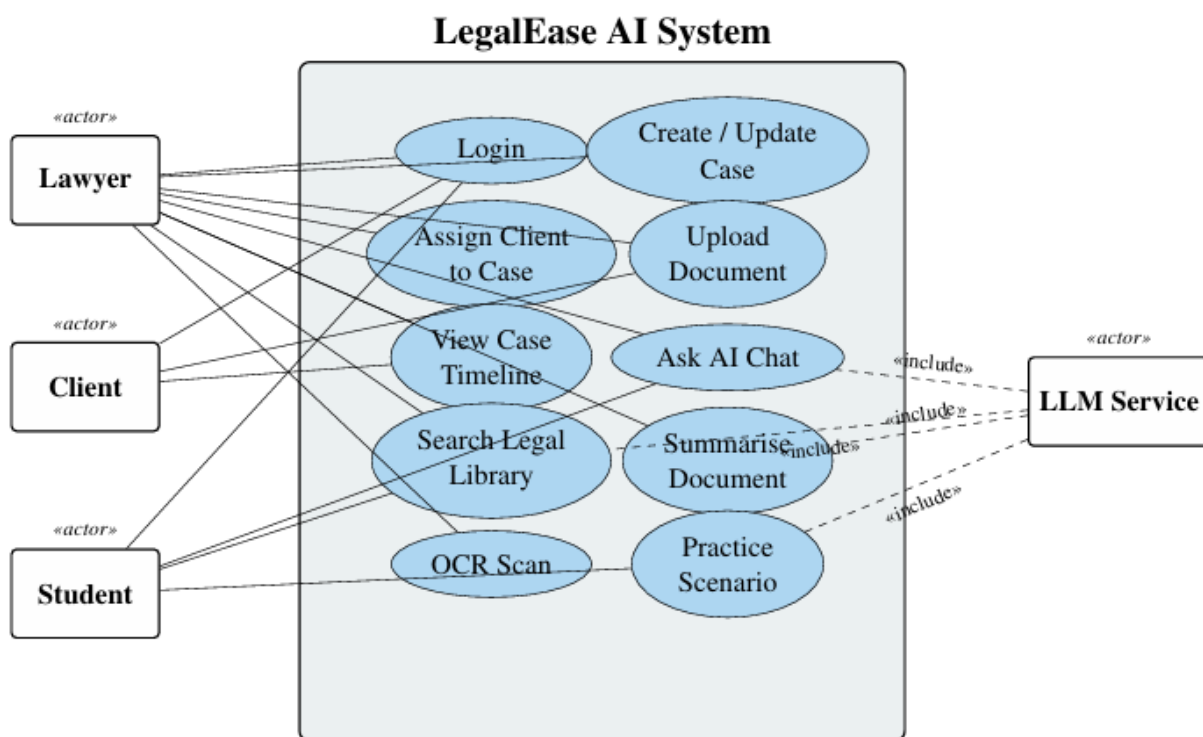


Figure 2.1: Use Case Diagram

2.2 Detailed Use Cases

2.2.1 UC-01: Login

Field	Description
ID	UC-01
Name	Login
Short Description	The user logs into the system using a registered email and password.
Goal	Authenticate the user and provide secure access to the dashboard.
Preconditions	The user has a registered account with a verified email.
Success End Condition	The user receives JWT access and refresh tokens and is redirected to the dashboard.
Fall End Condition	Login fails due to invalid credentials or account lock after multiple failed attempts.
Stakeholder	Lawyer / Client / Student
Trigger	The user enters email and password on the login page and presses the Login button.
Normal Flow	1. User submits email and password to POST /auth/login. 2. System checks the password using bcrypt against the stored hash. 3. System issues an access token (60 min) and a refresh token (7 days). 4. System writes LOGIN_OK to the audit log. 5. Client stores the tokens and redirects the user to the role's dashboard.
Alternative Flows	2a. Wrong password: failure counter increases and HTTP 401 is returned. 2b. Account locked after 5 failed attempts: HTTP 423 is returned.
Includes	Authentication, Audit Logging
Frequency of Use	Frequently used by all system users.
Constraints and Special Requirements	Passwords must be securely hashed using bcrypt. JWT tokens must be securely generated and stored securely.
Assumptions	The user has internet access and a valid registered account.
Notes and Issues	Login process should complete within 1 second and audit logging must always be available.

Table 2.1: Use Case UC-01: Login

2.2.2 UC-02: Create New Case

Field	Description
ID	UC-02
Name	Create New Case
Short Description	A Lawyer creates a new legal case in the system by entering case details.
Goal	Allow Lawyers to register and manage new legal cases.
Preconditions	The Lawyer is logged into the system with a valid account.
Success End Condition	A new case is successfully created and displayed on the dashboard.
Fall End Condition	The case is not created and validation errors are displayed.
Stakeholder	Lawyer
Trigger	The Lawyer clicks the “Create New Case” button.
Normal Flow	1. The Lawyer opens the dashboard and selects “Create New Case”. 2. The system displays the case creation form. 3. The Lawyer enters case details such as title, court, and type. 4. The Lawyer submits the form. 5. The system validates the information and stores the case. 6. The system creates an audit log entry. 7. The new case appears on the dashboard.
Alternative Flows	If required fields are empty, the system highlights validation errors. If the server is unavailable, the system shows an error message.
Includes	Login
Frequency of Use	About 15–20 times per day
Constraints and Special Requirements	Case titles must be unique for each Lawyer account.
Assumptions	The Lawyer has verified credentials.
Notes and Issues	Audit logging must occur on successful case creation.

Table 2.2: Use Case UC-02: Create New Case

2.2.3 UC-03: Assign Client to Case

Field	Description
ID	UC-03
Name	Assign Client to Case
Short Description	A Lawyer assigns a registered Client to an existing case.
Goal	Allow Clients to access case details and shared documents.
Preconditions	The Lawyer is authenticated and the Client account exists.
Success End Condition	The Client is linked to the case and receives a notification.
Fall End Condition	Assignment fails and the case remains unchanged.
Stakeholder	Lawyer
Trigger	The Lawyer selects “Assign Client” from a case page.
Normal Flow	1. The Lawyer opens an existing case. 2. The Lawyer clicks “Assign Client”. 3. The system displays a Client search option. 4. The Lawyer selects a Client account. 5. The Lawyer confirms the assignment. 6. The system links the Client to the case. 7. The system updates the case status and sends a notification.
Alternative Flows	If the Client is already assigned, the system displays a warning. If the Client account does not exist, the Lawyer is prompted to invite them.
Includes	Login, Send Notification
Frequency of Use	About 10–15 times per day
Constraints and Special Requirements	Only CLIENT role users can be assigned.
Assumptions	The Client has a registered account.
Notes and Issues	Unauthorized users must receive HTTP 403.

Table 2.3: Use Case UC-03: Assign Client to Case

2.2.4 UC-04: Ask Legal Question via AI Chat

Field	Description
ID	AI-01
Name	Ask Legal Question via AI Chat
Short Description	A user types a legal question in English or Urdu. The AI Chat Assistant retrieves relevant passages from the legal library using RAG and returns a cited answer.
Goal	Provide accurate, citation-backed answers to legal questions using stored Pakistani statutes and court judgments.
Preconditions	The user is authenticated and has an active chat session. The FAISS vector index contains Pakistani legal documents and the LLM service is available.
Success End Condition	The system returns a complete answer within 15 seconds along with inline citations to statutes or judgments.
Fall End Condition	If no relevant passage is found, the system refuses to answer. If the AI service is unavailable, an error message is displayed.
Stakeholder	Lawyer / Student
Trigger	The user enters a legal question in the chat box and presses the Send button.
Normal Flow	1. User opens the AI Chat module. 2. User types a legal question in English or Urdu. 3. System sends the query to the backend service. 4. The RAG service embeds the question using Sentence Transformers. 5. The FAISS index searches for the most relevant legal passages. 6. Retrieved passages are combined into a prompt. 7. The prompt is sent to the LLM service. 8. The system receives and formats the AI response with citations. 9. The answer is displayed and stored in chat history. 10. User may open citation links or continue the conversation.
Alternative Flows	4a. If no relevant passage exists, the system responds that no answer could be found. 4b. If the question is written in Urdu, the system responds in Urdu. 4c. If the AI service fails, the system displays an unavailable service message.
Includes	Login
Frequency of Use	Approximately 100–150 times per day.
Constraints and Special Requirements	Chat history is limited to the last 50 exchanges. AI response time must remain below 15 seconds for uncached queries.
Assumptions	The legal knowledge base contains indexed Pakistani statutes and judgments.
Notes and Issues	Language detection accuracy should remain above 95% for bilingual queries.

Table 2.4: Use Case UC-04: Ask Legal Question via AI Chat

2.2.5 UC-05: Upload and Analyse Legal Document

Field	Description
ID	DOC-01
Name	Upload and Analyse Legal Document
Short Description	A Lawyer uploads a legal document and the system generates an AI-powered summary along with highlighted key clauses.
Goal	Allow Lawyers to upload, store, and automatically analyse legal documents within a case.
Preconditions	The Lawyer is authenticated and has an active case. The NLP engine and Document Analysis service are available. The uploaded file is PDF, DOCX, or TXT and does not exceed 20 MB.
Success End Condition	The document is stored successfully, a summary is generated, and important clauses are identified and displayed.
Fall End Condition	If the file type is unsupported, the upload fails. If the NLP engine fails, a partial result with retry option is shown.
Stakeholder	Lawyer
Trigger	The Lawyer clicks “Upload Document” and selects a legal document file.
Normal Flow	1. Lawyer opens the case Documents tab. 2. Lawyer selects and uploads a file. 3. System validates file type and size. 4. File is stored securely in blob storage. 5. System creates a document metadata record. 6. Analysis service reads and processes the document. 7. AI generates a summary and identifies key clauses. 8. Analysis results are stored in the database. 9. Document appears in the case with status “Completed”. 10. Lawyer may optionally assign a category or label to the document.
Alternative Flows	3a. Unsupported file types are rejected with an error message. 3b. Files larger than 20 MB are rejected. 3c. If the NLP engine is unavailable, the analysis is queued for retry.
Includes	Login
Frequency of Use	Approximately 30–40 uploads per day.
Constraints and Special Requirements	Maximum upload size is 20 MB. Supported formats are PDF, DOCX, and TXT. AI summary must remain between 5–10% of the original content length.
Assumptions	The NLP model and blob storage service are operational.
Notes and Issues	Analysis confidence score should remain above 0.8 for high-confidence results.

Table 2.5: Use Case UC-05: Upload and Analyse Legal Document

2.2.6 UC-06: Search Legal Library by Semantic Query

Field	Description
ID	RES-01
Name	Search Legal Library by Semantic Query
Short Description	A Lawyer or Student enters a natural-language legal query and the AI Legal Research module returns ranked semantic search results from Pakistani statutes and court judgments.
Goal	Enable users to search legal documents using semantic understanding instead of keyword matching.
Preconditions	The user is authenticated. The FAISS vector index is populated with embeddings of Pakistani legal documents and the search service is available.
Success End Condition	The system returns up to 10 ranked results within 2 seconds, including document title, excerpt, court, year, and relevance score.
Fall End Condition	If no relevant result exists, the system displays a message asking the user to refine the query. If the search service fails, an error message is shown.
Stakeholder	Lawyer / Student
Trigger	The user enters a query in the AI Legal Research module and clicks the Search button.
Normal Flow	1. User opens the AI Legal Research module. 2. User enters a natural-language legal query. 3. System converts the query into embeddings using Sentence Transformers. 4. FAISS performs semantic similarity search on indexed legal documents. 5. Top-10 most relevant passages are retrieved and ranked. 6. System displays the results with document title, court, year, excerpt, and relevance score. 7. User may open any result to read the complete statute or judgment. 8. User may optionally save results to research notes or view similar cases.
Alternative Flows	2a. If the query is vague, the system suggests refined search queries. 2b. If results take longer than 2 seconds, a loading indicator is displayed. 2c. If no relevant result is found, the system displays a “No results found” message.
Includes	Login
Frequency of Use	Approximately 50–80 searches per day.
Constraints and Special Requirements	Search results must be returned within 2 seconds for 95% of queries under standard load conditions.
Assumptions	The legal library contains indexed Pakistani statutes and Supreme Court or High Court judgments.
Notes and Issues	Mean Reciprocal Rank for evaluation queries should remain above 0.60.

Table 2.6: Use Case UC-06: Search Legal Library by Semantic Query

2.3 Functional Requirements

Functional requirements are grouped by module. Each requirement has an ID, a description, and an acceptance criterion that testing will verify.

2.3.1 2.2.1 Case Management Module

Following are the requirements for module 1:

1. **FR-CM01:** Allow a Lawyer to create a case with a title, description, court, type, and opening date.
Acceptance Criterion: POST to /cases returns 201; the case shows up on the lawyer's dashboard within 1s
2. **FR-CM02:** Allow a Lawyer to assign Clients with role-specific permissions.
Acceptance Criterion: The assigned Client can view the case; non assigned users get HTTP 403.
3. **FR-CM03:** Allow authorised users to upload PDF, DOCX, or image files (max 20MB) to a case.
Acceptance Criterion: Upload succeeds, the file is stored, and the metadata appears in the case's document list.
4. **FR-CM04:** Record hearings (date, time, venue) and show them on the case timeline.
Acceptance Criterion: Hearings sorted by date; future hearings highlighted.
5. **FR-CM05:** Show a timeline of every event on a case.
Acceptance Criterion: Timeline displays creation, assignment, document upload, hearing, and status changes in order
6. **FR-CM06:** Update case status using a fixed state machine: Created → Assigned → InProgress → HearingScheduled → Closed.
Acceptance Criterion: Invalid transitions return HTTP 409; valid ones update the case and add an audit log entry
7. **FR-CM07:** Let a Client view but not edit their assigned cases.
Acceptance Criterion: Client sees only their own cases; PUT/DELETE return HTTP 403

2.3.2 AI Legal Chat Assistant Module

Following are the requirements for module 2:

1. **FR-AI-01:** Accept legal questions in English or Urdu.
Acceptance Criterion: Language detection accuracy $\geq 95\%$ on a 100-question bilingual test set.

2. **FR-AI-02:** Find the top 5 most relevant passages from the legal library before answering.
Acceptance Criterion: Retrieval recall@5 ≥ 0.80 on a 50-pair evaluation set.
3. **FR-AI-03:** Refuse to answer if no passage has similarity above 0.7.
Acceptance Criterion: On 30 out-of-library questions, $\geq 90\%$ are correctly refused.
4. **FR-AI-04:** Show inline citations to the source statute or judgment with every answer.
Acceptance Criterion: At least one citation per answered response; no broken links.
5. **FR-AI-05:** Keep the last 50 chat exchanges per user per case.
Acceptance Criterion: The history endpoint returns the last 50 in ≤ 500 ms.

2.3.3 Document Analysis Module

Following are the requirements for module 3:

1. **FR-DOC-01:** Accept PDF, DOCX, or plain-text files.
Acceptance Criterion: Supported formats upload fine; others return HTTP 415.
2. **FR-DOC-02:** Produce a short summary of an uploaded document.
Acceptance Criterion: Summary is 5–10% of source length; ROUGE-L ≥ 0.30 on 20 documents.
3. **FR-DOC-03:** Mark key clauses (parties, jurisdiction, term, indemnity, termination) in contracts.
Acceptance Criterion: Precision ≥ 0.80 and recall ≥ 0.75 on 25 labelled contracts.
4. **FR-DOC-04:** Flag missing clauses by comparing against a template.
Acceptance Criterion: Missing-clause list is correct on 10 test contracts.

2.3.4 AI Legal Research Module

Following are the requirements for module 4:

- (a) **FR-RES-01:** Provide semantic search over statutes and judgments.
Acceptance Criterion: Mean Reciprocal Rank ≥ 0.60 on the labelled evaluation set.
- (b) **FR-RES-02:** Allow filtering by court, year, and case type.
Acceptance Criterion: Filters narrow the result set correctly.

- (c) **FR-RES-03:** Return search results in $\leq 2s$ for 95% of queries.
Acceptance Criterion: Verified by a load test with k6.
- (d) **FR-RES-04:** Suggest similar past cases for any selected judgment.
Acceptance Criterion: At least 2 of the top 3 suggestions rated relevant by a domain expert.

2.3.5 Contract Drafting and Compliance Module

Following are the requirements for module 5:

- (a) **FR-CON-01:** Provide reusable templates (Employment, NDA, Service Agreement).
Acceptance Criterion: At least 3 templates available; PDF export works.
- (b) **FR-CON-02:** Suggest clauses while drafting.
Acceptance Criterion: On 20 drafting sessions, $\geq 70\%$ of suggestions rated useful.
- (c) **FR-CON-03:** Check a draft contract against compliance rules and produce a report.
Acceptance Criterion: Report shows passed and failed checks; verified on 15 test contracts.
- (d) **FR-CON-04:** Keep version history with diff support.
Acceptance Criterion: Versions stored with timestamp; diff renders within 1s.

2.3.6 OCR and Scanned-Document Module

Following are the requirements for module 6:

- (a) **FR-OCR-01:** Accept scanned PDFs and images (PNG, JPG).
Acceptance Criterion: Files up to 50MB process without error.
- (b) **FR-OCR-02:** Extract text with $\geq 90\%$ accuracy on English print and $\geq 80\%$ on Urdu.
Acceptance Criterion: Verified by character error rate on a 30 document benchmark.
- (c) **FR-OCR-03:** Save both the original scan and the extracted text.
Acceptance Criterion: Both rows visible in the document table.
- (d) **FR-OCR-04:** Pass extracted text to Document Analysis within 30s for documents up to 20 pages.
Acceptance Criterion: End-to-end latency measured by trace logs.

2.3.7 Practice Simulator Module

Following are the requirements for module 7:

- (a) **FR-PS-01:** Provide at least 10 different practice scenarios.
Acceptance Criterion: Catalogue has 10+ entries across civil, criminal, and commercial law.
- (b) **FR-PS-02:** Simulate a back-and-forth dialogue with a virtual client or judge.
Acceptance Criterion: Conversation stays coherent over 10+ turns in a manual review.
- (c) **FR-PS-03:** Give rubric-based feedback on the student's response.
Acceptance Criterion: Feedback covers legal accuracy, structure, and clarity.
- (d) **FR-PS-04:** Track student progress across sessions.
Acceptance Criterion: Progress visible at `/students/{id}/progress`.

2.3.8 Notifications Module

Following are the requirements for module 8:

- (a) **FR-NOT-01:** Send an in-app notification when a hearing is scheduled or rescheduled.
Acceptance Criterion: Notification appears within 5s of the change.
- (b) **FR-NOT-02:** Send an email reminder 24 hours before each hearing.
Acceptance Criterion: Email accepted by SMTP relay.
- (c) **FR-NOT-03:** Mark notifications as read on click and keep the read state.
Acceptance Criterion: Read state survives a page reload.

2.4 Non-Functional Requirements

Every NFR below has a measurable target, a verification method, and an acceptance threshold.

2.4.1 Reliability

Following are the reliability non-functional requirements:

1. **REL-01:** Core API (auth, cases, documents) maintains $\geq 99\%$ uptime per month.
Verification Method: UptimeRobot health checks over 30 days.
Acceptance Threshold: $\geq 99.0\%$.

2. **REL-02:** Full database backup runs every 24h and is kept for 7 days.
Verification Method: Inspect backup logs and restore one backup per release.
Acceptance Threshold: 100% backups complete; restore works.
3. **REL-03:** After a process crash, the system resumes serving in ≤ 5 min with no committed-transaction loss.
Verification Method: Chaos test: kill the API container, time recovery, run consistency check.
Acceptance Threshold: Recovery ≤ 5 min, zero data loss.
4. **REL-04:** If the AI service fails, core case management still works and AI returns a clear error.
Verification Method: Stop the AI worker, exercise the UI.
Acceptance Threshold: Core works; AI errors within 3s.

2.4.2 Usability

Following are the usability non-functional requirements:

1. **USE-01:** A user can create a case from the dashboard in ≤ 3 clicks.
Verification Method: UI walkthrough with 5 first-time users.
Acceptance Threshold: $\geq 4/5$ users finish in ≤ 3 clicks.
2. **USE-02:** A first-time user can finish core tasks (login, create case, upload, ask AI) in ≤ 10 minutes.
Verification Method: Usability study with 5 participants.
Acceptance Threshold: $\geq 4/5$ finish in 10 min.
3. **USE-03:** The interface is responsive on desktop, tablet, and mobile.
Verification Method: Visual regression tests at three breakpoints.
Acceptance Threshold: 100% pages render without overflow.
4. **USE-04:** Every error message has a description and a fix hint.
Verification Method: Inspect all 4xx/5xx responses and toasts.
Acceptance Threshold: 100% of errors have both fields.
5. **USE-05:** The platform scores ≥ 70 on the System Usability Scale.
Verification Method: Post-task SUS questionnaire with 8 users.
Acceptance Threshold: SUS ≥ 70 .

2.4.3 Performance

Following are the performance non-functional requirements:

1. **PER-01:** 95% of dashboard/case-list pages load in $\leq 2s$ on a 20 Mbps line.
Verification Method: Lighthouse + k6 load test of /cases at 50 users.
Acceptance Threshold: p95 $\leq 2s$.
2. **PER-02:** The AI Chat returns a complete answer within 5s (cached) or 15s (uncached).
Verification Method: End-to-end timing on 100 queries.
Acceptance Threshold: p95 within bounds.
3. **PER-03:** Document upload of files up to 20MB finishes in $\leq 10s$ on 20 Mbps.
Verification Method: Synthetic upload benchmark.
Acceptance Threshold: p95 $\leq 10s$.
4. **PER-04:** The system handles 100 concurrent users with error rate $\leq 1\%$.
Verification Method: Locust load test: 100 users, 30 minutes.
Acceptance Threshold: Error rate $\leq 1\%$, p95 $\leq 3s$.
5. **PER-05:** Database queries on cases and documents return in $\leq 200ms$ (p95).
Verification Method: pgBadger query analysis on staging.
Acceptance Threshold: p95 $\leq 200ms$.

2.4.4 Security

Following are the security non-functional requirements:

1. **SEC-01:** Passwords are stored with bcrypt (cost ≥ 12).
Verification Method: Code review and database inspection.
Acceptance Threshold: 100% of password_hash values match bcrypt format.
2. **SEC-02:** All HTTP traffic uses TLS 1.2 or higher.
Verification Method: SSL Labs scan.
Acceptance Threshold: Grade $\geq A$.
3. **SEC-03:** Role-based access control on every protected endpoint.
Verification Method: Automated RBAC test for all 3 roles.
Acceptance Threshold: 100% endpoints covered, zero unauthorised access.
4. **SEC-04:** Every privileged action (login, role change, upload, status change) is logged.
Verification Method: Inspect audit_log after each action.
Acceptance Threshold: 100% of actions produce a log row.

5. **SEC-05:** The system has zero critical or high OWASP Top 10 issues.

Verification Method: OWASP ZAP active scan in CI.

Acceptance Threshold: Zero critical/high findings.

6. **SEC-06:** JWT access tokens expire in ≤ 60 min; refresh tokens in ≤ 7 days.

Verification Method: Inspect JWT exp claim.

Acceptance Threshold: Within stated bounds.

Chapter 3

System Overview

This chapter explains how LegalEase AI is built. It covers the architectural design, the data design, and the design models for the iterations completed in FYP-1.

3.1 Architectural Design

LegalEase AI uses a Layered Architecture. The system is split into three layers, with each layer communicating only with the layer directly below it:

1. **Presentation Layer**– the user interface (React 19 web app [11]).
2. **Application Layer**– business logic and APIs (FastAPI backend [18]).
3. **Data Layer**– persistent storage (PostgreSQL, blob storage, and FAISS vector index).

A layered architecture suits this project for three reasons. First, each layer has a single responsibility and can be developed and tested independently. Second, changes in one layer do not force changes in the others, keeping maintenance cost low. Third, the system operates within one bounded context (legal case management), so a microservices decomposition would add overhead without benefit. Layered architecture is the established pattern for web applications of this size .

3.1.1 High Level Box and Line Diagram

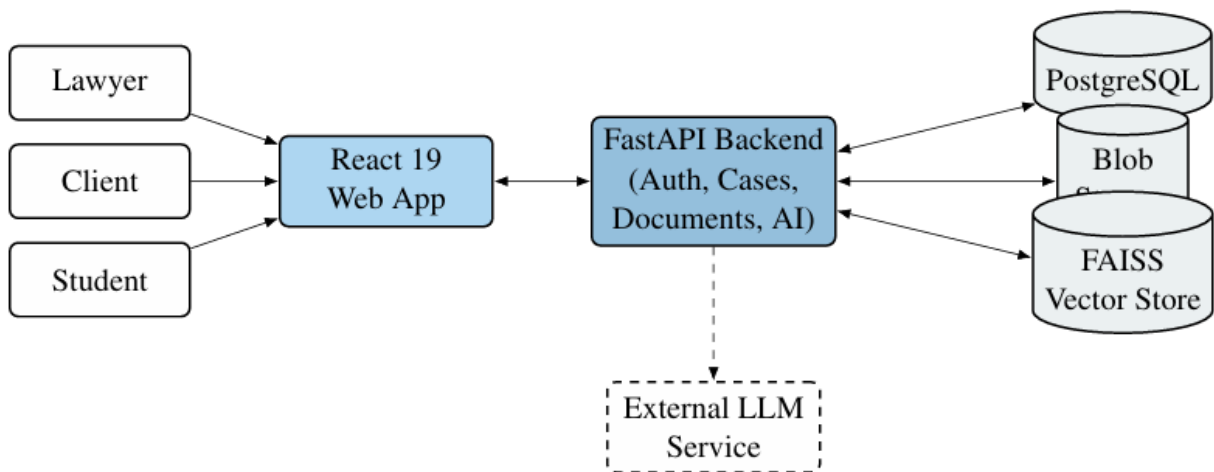


Figure 3.1: Box and Line Diagram

3.1.2 Architecture Diagram

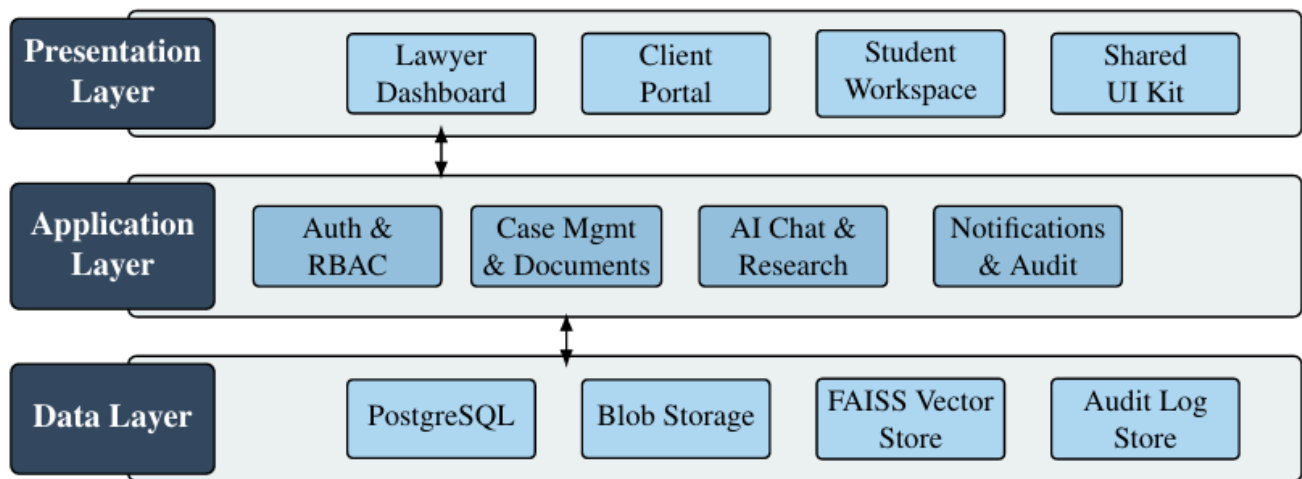


Figure 3.2: System Architecture Diagram

3.2 Data Design

The LegalEase AI system stores and manages data using multiple specialized storage mechanisms to ensure scalability, security, and efficient retrieval.

1. **Structured data.** Core system entities such as users, lawyers, clients, students, cases, documents metadata, deadlines, chat sessions, chat messages, simulation attempts, notifications, and audit logs are stored in PostgreSQL [16]. The database schema uses primary keys, foreign keys, UNIQUE constraints, and ENUM validations to maintain referential integrity and prevent invalid records from being inserted.
2. **Document files.** Uploaded legal documents including PDFs, DOCX files, petitions, judgments, notices, and scanned images are stored in external object/blob storage rather than directly inside the relational database. Only file metadata and storage paths are maintained in PostgreSQL. This reduces database size and improves file-serving performance through secure signed URLs
3. **Vector embeddings.** Legal case precedents, statutes, and processed document text are converted into semantic vector embeddings using transformer-based NLP models [5]. These embeddings are stored in a vector database/index to support semantic search, Retrieval-Augmented Generation (RAG), AI Chat, and AI Legal Research functionalities. Each embedding entry maintains a reference to its corresponding PostgreSQL record to preserve citation accuracy and traceability.
4. **AI-generated outputs** such as document summaries, detected clauses, risk assessments, OCR extracted text, and simulation feedback reports are stored in structured JSON/JSONB fields within PostgreSQL. This enables flexible storage of semi-structured AI responses while maintaining relational consistency.
5. **AI analysis data:** AI-generated outputs such as document summaries, detected clauses, risk assessments, OCR extracted text, and simulation feedback reports are stored in structured JSON/JSONB fields within PostgreSQL. This enables flexible storage of semi-structured AI responses while maintaining relational consistency.
6. **Audit and security data:** Every sensitive or privileged system action is recorded in the activitylogs table for accountability and monitoring purposes. User passwords are never stored in plain text and are securely hashed using bcrypt or equivalent encryption mechanisms. Authentication tokens and session-related security data are managed using secure JWT-based authentication practices

3.3 Domain Model

The domain model is a high-level conceptual representation of the LegalEase AI system. It describes the main entities of the system, their attributes, and the relationships between them. The purpose of the domain model is to provide a clear understanding of the business domain and how different parts of the system interact with each other before actual implementation begins.

A domain model usually contains the following components:

- **Entities:** These are the main objects of the system that represent real-world concepts. In the LegalEase AI system, important entities include User, Lawyer, Client, Student, Case, Document, Deadline, ChatSession, and SimulationScenario. Each entity stores important information related to the legal case management process.
- **Attributes:** Attributes are the properties that describe an entity. For example, the Case entity contains attributes such as caseId, title, description, caseType, and status. Similarly, the Document entity includes attributes like fileName, fileType, summaryText, and uploadedAt.
- **Relationships:** TRelationships show how entities are connected with each other inside the system. For example, a Lawyer can manage multiple Cases, a Case can contain many Documents, and a ChatSession can have multiple ChatMessages. These relationships help explain how data flows within the application.
- **Associations:** Associations define the interaction and dependency between entities. For instance, a CaseParticipant associates a User with a Case and defines their role such as lawyer, client, witness, or judge. Similarly, SimulationAttempt connects a Student with a SimulationScenario for legal practice and evaluation.

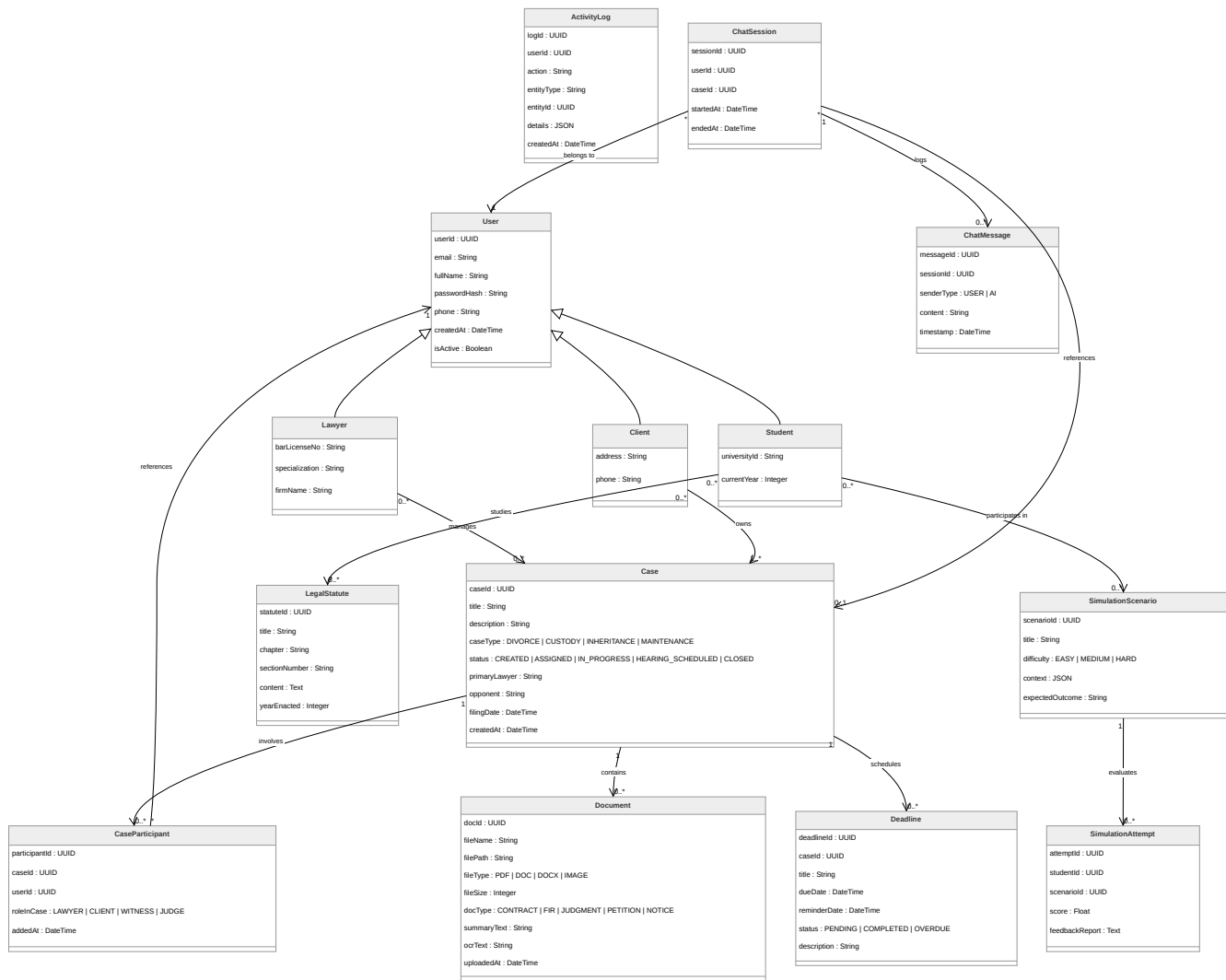


Figure 3.4: Domain model Diagram

3.4 Design Models

This section presents the design diagrams for the iterations completed in FYP-1. All diagrams follow standard UML notation [2].

Design Models for Object Oriented Development Approach

The applicable models for the project using object oriented development approach are:

- Activity Diagram

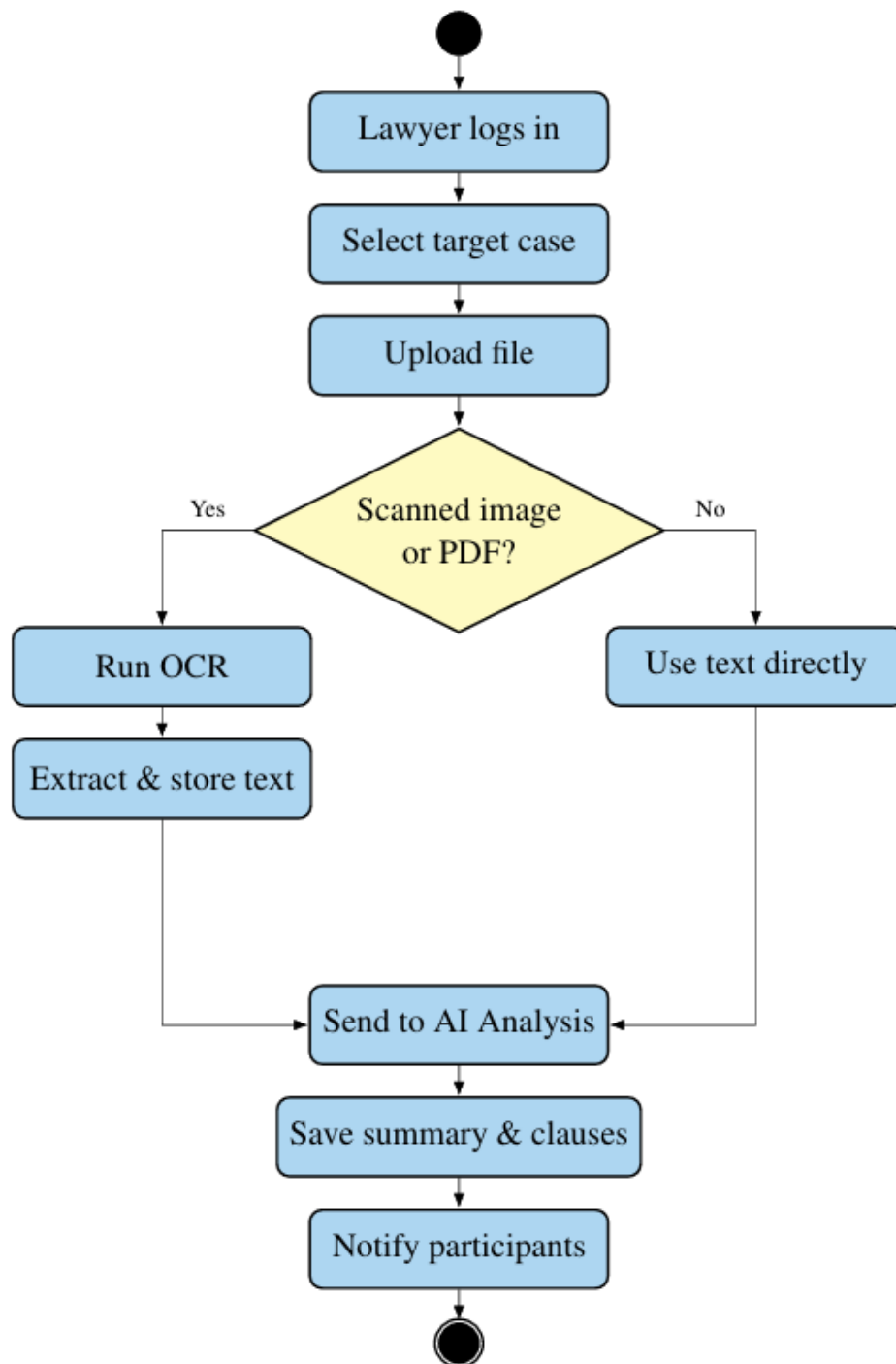


Figure 3.5: Activity Diagram

- **Class Diagram**

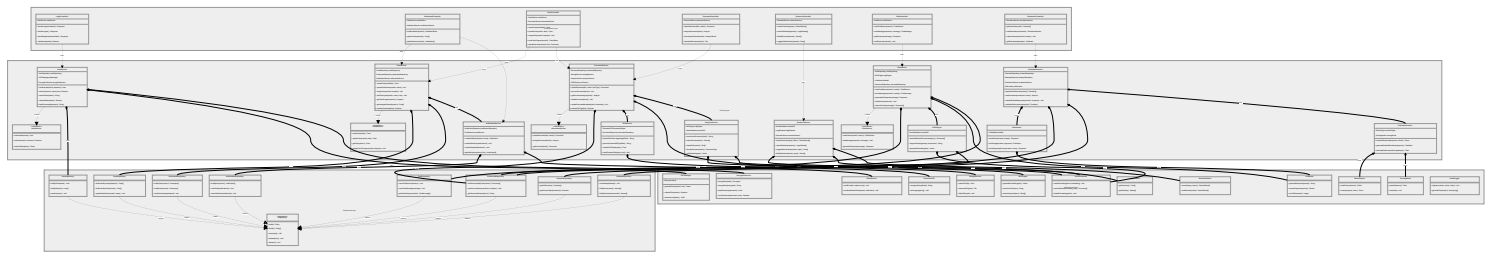


Figure 3.6: Class Diagram

- **Class-level Sequence Diagram**

SD-1: User Login

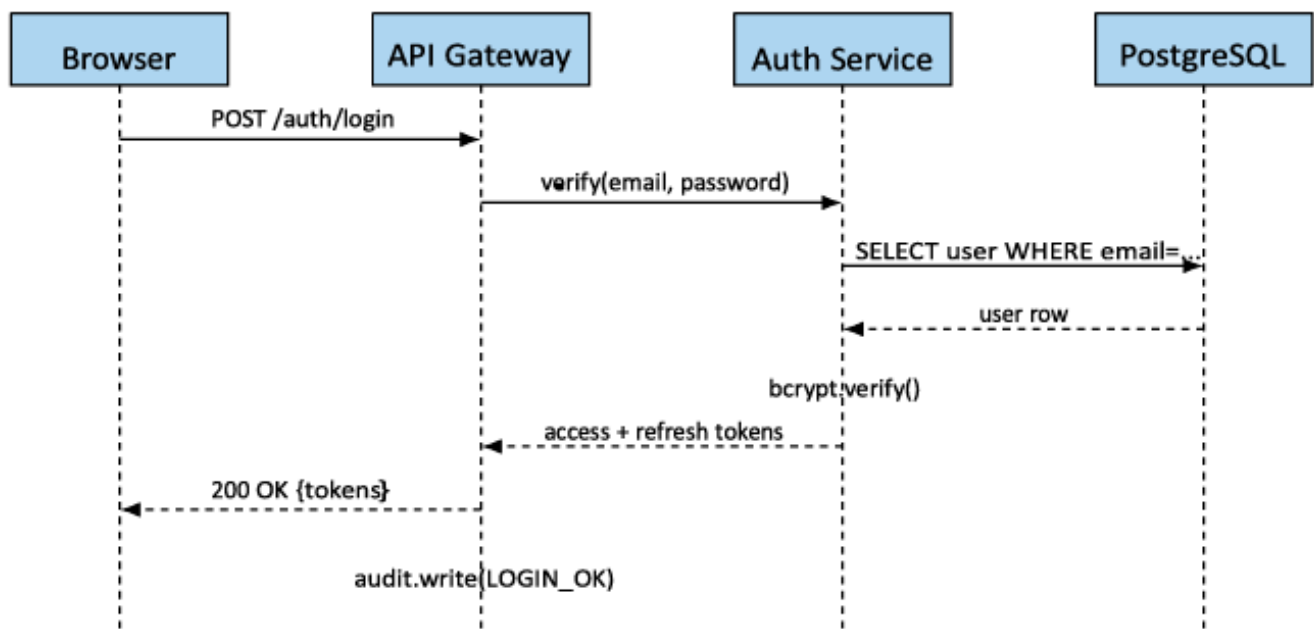


Figure 3.7: SD User Login Diagram

SD-2: AI Chat with RAG

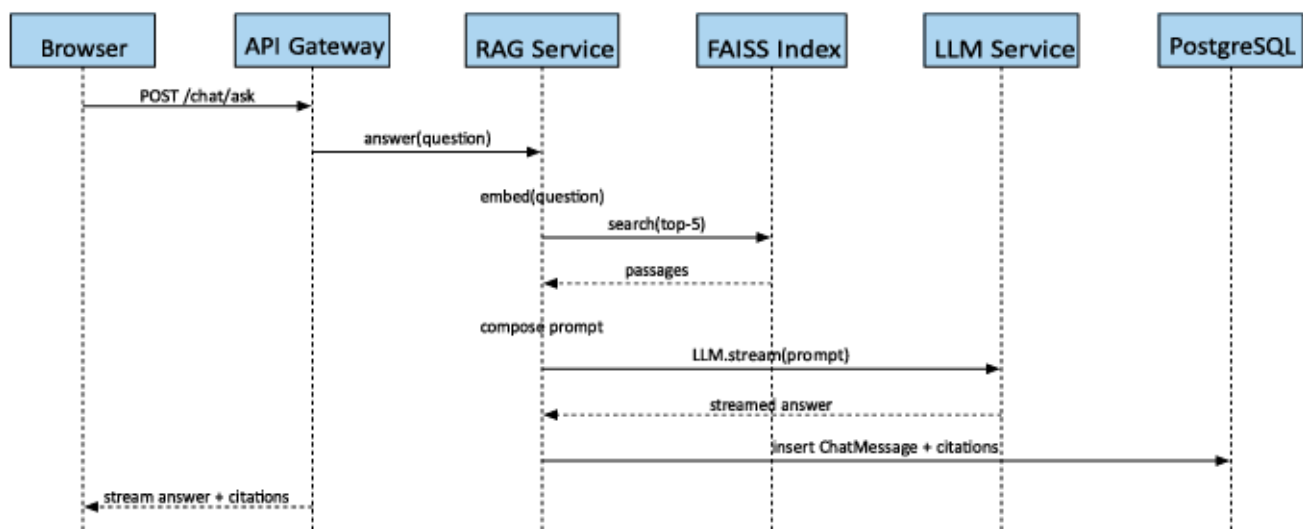


Figure 3.8: SD-2 AI Chat with Rag

SD-3: Upload Document with OCR

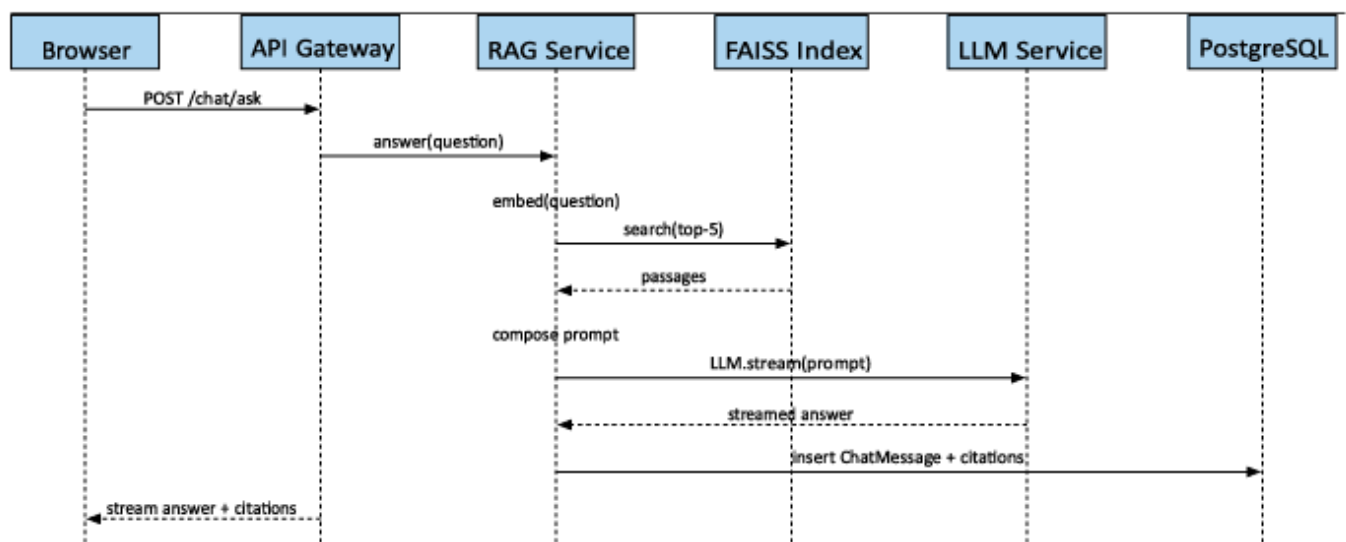


Figure 3.9: SD Upload Document with OCR

Chapter 4

Implementation and Testing

This chapter describes the work delivered in FYP-1, organised by iteration. Iteration 1 delivered Role-Based Authentication and the Case Management Module. Iteration 2 delivered the AI Chatbot (RAG-based Legal Chat Assistant), the AI Legal Research Module, and the OCR Module. The algorithms for each iteration are presented first, followed by the external APIs and SDKs used, and then the unit-testing results.

4.1 Algorithm Design

Iteration 1: Role-Based Authentication and Case Management

Authentication and JWT Issuance

Authentication uses JSON Web Tokens (JWT). On a successful login, the server verifies the bcrypt password hash and issues an access token (60 minutes) and a refresh token (7 days). Role-based access control runs as a FastAPI dependency that reads the role claim from the token and rejects any request whose role does not match the required route permission.

Algorithm 1 Login and Issue JWT Pair

```
1: procedure LOGIN(email, password)
2: user ← LOADUSERBYEMAIL(email)
3: if user = ⊥ or user.locked then
4: return {401, "Invalid credentials"}
5:   end if
6: if not BCrypt.VERIFY(password, user.passwordHash) then
7: INCREMENTFAILURES(user)
8: return {401, "Invalid credentials"}
9:   end if
10: access ← JWT.SIGN({sub : user.id, role : user.role, exp : now+60m})
11: refresh ← JWT.SIGN({sub : user.id, kind : "r", exp : now+7d})
12: AUDITLOG.WRITE("LOGIN_OK", user.id)
13: return {200, {access, refresh}}
14: end procedure
```

Figure 4.1: Login and Issue JWT Pair

Case Management and Status Transitions

Cases follow a fixed lifecycle enforced by a finite-state machine. The database enforces the valid status values through a CHECK constraint; the service layer then guards individual transitions so that no invalid state change can be persisted.

Allowed transitions:

allowed = {Created→{Assigned}, Assigned→{InProgress}, InProgress→{HearingScheduled, Closed},
HearingScheduled→{InProgress}}

```

1: procedure UPDATECASESTATUS(caseId, user, newStatus)
2:   c ← CASE.LOAD(caseId)
3:   if user.role ≠ Lawyer or user.id ∉ c.lawyerIds then
4:     raise HTTP 403
5:   end if
6:   if newStatus ∉ allowed[c.status] then
7:     raise HTTP 409 "Illegal status transition"
8:   end if
9:   c.status ← newStatus
10:  CASE.SAVE(c)
11:  AUDITLOG.WRITE("CASE_STATUS_CHANGE", user.id, c.id, newStatus)
12: end procedure

```

Figure 4.2: Validated Case Status Transition

Upload with SHA-256 Deduplication

Documents are streamed to blob storage as bytes arrive. A SHA-256 hash is computed during the upload for deduplication. The metadata row is written to PostgreSQL only after the upload completes, so a network failure mid-stream cannot leave an orphan blob.

Algorithm 3 Stream-and-Commit Document Upload

```
1: procedure UPLOADDOCUMENT(caseld, user, stream)
2: Authorise that user may write to caseld
3: tmpKey ← "tmp/" + uuid()
4: tmp ← BLOB.OPENWRITE(tmpKey)
5:   hasher ← SHA256.NEW
6: while chunk ← READ(stream) do
7:   BLOB.WRITE(tmp, chunk)
8:   HASHER.UPDATE(hasher, chunk)
9:   end while
10:  digest ← HASHER.FINALISE(hasher)
11: if BLOB.SIZE(tmp) > 20MB then
12:   BLOB.DELETE(tmp); raise HTTP 413
13:   end if
14: blobKey ← BLOB.MOVETo(tmp, "cases/" + caseld + "/" + digest)
15: row ← DOCUMENT.INSERT(caseld, blobKey, digest, user.id)
16: AUDITLOG.WRITE("DOC_UPLOAD", user.id, row.id)
17:   return row
```

Figure 4.3: Stream-and-Commit Document Upload

Iteration 2: AI Chatbot, AI Legal Research, and OCR**Legal Library Index Construction**

The legal library is built from the Pakistan Code [4] and reported judgments of the Peshawar [15] and Sindh [19] High Courts. Each document is split into 800-token chunks with a 100-token overlap. Chunks are embedded with a multilingual sentence-transformer [5] and stored in a FAISS IndexFlatIP after L2-normalisation, so cosine similarity equals the inner product.

Algorithm 4 Build the Legal Library Index

```

1: procedure INGESTCORPUS(documents)
2:   for each doc in documents do
3: text ← EXTRACTTEXT(doc)
4: chunks ← CHUNKBYTOKENS(text, size=800, overlap=100)
5:   for each c in chunks do
6: row ← LEGALCORPUSENTRY.INSERT(doc.id, c.span, c.text, doc.title, doc.url)
7: vec ← EMBED(c.text)
8: NORMALISE(vec)
9: FAISS.ADD(vec, id=row.id)
10:   end for
11: end for
12: FAISS.SAVEINDEX("legal_corpus.faiss")
13: end procedure

```

Figure 4.4: Build the Legal Library Index

RAG Answer Pipeline (AI Chatbot)

The AI Chatbot uses Retrieval-Augmented Generation (RAG) [9]: the system retrieves the most relevant passages from the index, builds a prompt, calls the LLM, and returns the answer with citations. The threshold check on line 5 implements FR-AI-03: if no passage scores above 0.7, the system refuses to answer instead of guessing.

Algorithm 5 RAG Answer Generation

```
1: procedure ANSWER(user, caseld, question)
2: lang ← DETECTLANGUAGE(question)
3: qVec ← EMBED(question)
4: NORMALISE(qVec)
5: hits ← FAISS.SEARCH(qVec, k=5)
6: hits ← {h ∈ hits | h.score ≥ 0.7}
7:   if hits = 0/ then
8:     return "No authoritative source found."
9:   end if
10: passages ← LEGALCORPUSENTRY.LOADBYIDS({h.rowId | h ∈ hits})
11: history ← CHATSESSION.LASTN(user.id, caseld, n=10)
12: prompt ← COMPOSEPROMPT(question, passages, history, lang)
13:   answer ← LLM.STREAMCOMPLETE(prompt)
14: citations ← [(p.title, p.url, p.span) | p ∈ passages]
15: CHATMESSAGE.INSERT(user.id, caseld, question, answer, citations)
16:   return (answer, citations)
17: end procedure
```

Figure 4.5: RAG Answer Generation

4.2 External APIs/SDKs

Table 4.1: External APIs / SDKs Used in FYP-1

API / SDK	Purpose	Where Used	Endpoint / Class
OpenAI Chat Completions [12]	LLM answer generation	AI Chatbot	POST /v1/chat/completions
Sentence Transformers [5]	Multilingual embeddings	Index, RAG, Research	SentenceTransformer.encode()
FAISS [6]	Vector similarity search	RAG & Research retrieval	IndexFlatIP.search()
Tesseract OCR [3]	Text extraction from scanned documents	OCR Module	pytesseract.image_to_string()
PostgreSQL [16]	Relational database storage	Backend (all modules)	SQLAlchemy ORM
PyJWT [7]	Sign and verify JWT tokens	Authentication middleware	jwt.encode() / jwt.decode()
bcrypt	Password hashing	Authentication module	bcrypt.hashpw
Celery + Redis	Asynchronous task queue	OCR jobs	Celery.send_task

4.3 Testing Details

Testing Strategy Overview

Unit testing is conducted at the service layer to verify individual component functionality in isolation. Each service has corresponding test cases covering normal operations, edge cases, and error conditions.

Module 1: Authentication Service Tests

Table 4.2: Authentication Service Unit Tests

Test Case ID	Test Objective	Precondition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/Fail
UT-AUTH-001	Verify successful login with valid credentials	User account exists with email and password in database	1. Call <code>authenticate()</code> with valid email and password 2. Verify returned object 3. Check JWT token	Email: lawyer@legalease.com Password: SecurePass123	Returns User object with <code>userId</code> , role <code>LAWYER</code> , and JWT token generated. Status: 200 OK	User session created; JWT token valid for 24 hours	As Expected	Pass
UT-AUTH-002	Verify rejection of invalid password	User lawyer@legalease.com exists with correct password	1. Call <code>authenticate()</code> with wrong password 2. Verify error response 3. Check no token generated	Email: lawyer@legalease.com Password: WrongPassword	Returns <code>AuthenticationException</code> : "Invalid credentials". No JWT token. Status: 401 Unauthorized	User not authenticated; no session created	As Expected	Pass
UT-AUTH-003	Verify handling of non-existent user login	No account exists for non-existent@user.com	1. Call <code>authenticate()</code> with non-existent email 2. Verify error handling	Email: non-existent@user.com Password: anypassword	Returns <code>UserNotFoundException</code> : "User not found". Status: 404 Not Found	No session created	As Expected	Pass

Module 2: Case Management Service Tests

Table 4.3: Case Management Service Unit Tests

Test Case ID	Test Objective	Precondition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/Fail
UT-CASE-001	Verify successful case creation	Lawyer lawyer-001 authenticated. Client client-001 exists	1. Call <code>createCase()</code> 2. Verify case stored 3. Check lawyer assigned 4. Verify status <code>CREATED</code>	Title: Divorce Case – Ahmed vs Fatima Type: <code>DIVORCE</code> FilingDate: 2026-05-01	Returns Case object with generated <code>caseId</code> , status <code>CREATED</code> , and database record created. Status: 200 OK	Case visible in lawyer dashboard; client notified	As Expected	Pass
UT-CASE-002	Verify case assignment to lawyer	Case case-001 exists with status <code>CREATED</code> . Lawyer lawyer-002 is active	1. Call <code>assignCase()</code> 2. Verify status updated 3. Check <code>CaseParticipant</code> created 4. Verify notification sent	CaseId: case-001 LawyerId: lawyer-002	Case status updated to <code>ASSIGNED</code> . <code>CaseParticipant</code> created with role <code>LAWYER</code> , email sent to lawyer. Status: 200 OK	Lawyer can view case; case in worklist	As Expected	Pass
UT-CASE-003	Verify rejection of invalid case assignment	Lawyer lawyer-inactive is marked inactive in database	1. Call <code>assignCase()</code> with inactive lawyer 2. Verify validation	CaseId: case-001 LawyerId: lawyer-inactive	Returns <code>ValidationException</code> : "Lawyer is not active or not found". Case not updated. Status: 400 Bad Request	Case remains in previous status	As Expected	Pass

Module 3: Document Service Tests

Table 4.4: Document Service Unit Tests

Test Case ID	Test Objective	Precondition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/Fail
UT-DOC-001	Verify document upload and storage	Case case-001 exists. File available. Storage service running	1. Call <code>uploadDocument()</code> 2. Verify file stored in S3 3. Check database record 4. Verify metadata stored	FileName: <code>divorce_petition.pdf</code> FileSize: 150 KB DocType: PETITION CaseId: case-001	Returns Document object with generated docId, S3 file path, documentType PETITION. Status: 201 Created	Document visible in case; ready for analysis	As Expected	Pass
UT-DOC-002	Verify rejection of invalid file types	Case case-001 exists	1. Attempt upload of .exe file 2. Verify validation 3. Check file rejected	FileName: <code>malware.exe</code> FileType: EXE	Returns <code>ValidationException</code> : "File type not allowed. Allowed: PDF, DOC, DOCX, JPG, PNG". Status: 415 Unsupported Media Type	Case documents unchanged	As Expected	Pass
UT-DOC-003	Verify AI-based document summarisation	Document doc-001 uploaded. NLP engine and BERT model loaded	1. Call <code>summarizeDocument()</code> 2. Verify NLP engine call 3. Check summary stored 4. Verify confidence score	DocumentId: <code>doc-001</code>	Returns analysis object with summary (200-300 words), <code>confidenceScore</code> : 0.92, and key phrases. Status: 200 OK	Summary indexed for search and displayed in UI	As Expected	Pass

Module 4: AI Legal Chat Tests

Table 4.5: AI Legal Chat Service Unit Tests

Test Case ID	Test Objective	Precondition	Steps	Test Data	Expected Result	Post-condition	Actual Result	Pass/Fail
UT-CHAT-001	Verify chat session initialisation	User user-001 authenticated. Case case-001 exists	1. Call <code>startChatSession()</code> 2. Verify session created 3. Check session ID returned	UserId: user-001 CaseId: case-001	Returns <code>ChatSession</code> with generated <code>sessionId</code> , <code>userId</code> , <code>caseId</code> , and <code>startedAt</code> timestamp. Status: 200 OK	Chat interface ready for user input	As Expected	Pass
UT-CHAT-002	Verify RAG-based legal query processing	Chat session active. Case documents available. Vector DB initialised	1. Call <code>sendMessage()</code> with legal query 2. Verify document retrieval 3. Check prompt augmented 4. Verify LLM response	SessionId: <code>session-001</code> Query: "What are my rights in child custody?"	Returns AI <code>ChatMessage</code> with 3-5 paragraph response, retrieved source documents, response time under 5 seconds. Status: 200 OK	Response displayed to user; message logged	As Expected	Pass
UT-CHAT-003	Verify handling of off-topic queries	Chat session active	1. Send off-topic query 2. Verify out-of-scope detection 3. Check appropriate response	Query: "What's the best pizza recipe?"	Returns <code>ChatMessage</code> informing user the query is outside scope and redirecting to family law topics. <code>outOfScope</code> : true	User redirected to appropriate queries	As Expected	Pass

Bibliography

- [1] Adalat Online. Adalat Online: Legal Services Portal of Pakistan. <https://www.adalatonline.pk>, 2026. Accessed: 2026-04-15.
- [2] Grady Booch, James Rumbaugh, and Ivar Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley, 2nd edition, 2005.
- [3] Google. Tesseract OCR: Open Source OCR Engine. <https://github.com/tesseract-ocr/tesseract>, 2026. Accessed: 2026-04-12.
- [4] Government of Pakistan. Pakistan Code – Federal Laws of Pakistan. <http://www.pakistancode.gov.pk>, 2026. Accessed: 2026-04-10.
- [5] Hugging Face. Sentence-Transformers: Multilingual Embedding Models. <https://www.sbert.net>, 2026. Accessed: 2026-04-12.
- [6] Jeff Johnson, Matthijs Douze, and Hervé Jégou. FAISS: A Library for Efficient Similarity Search. <https://github.com/facebookresearch/faiss>, 2017.
- [7] Michael B. Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT). RFC RFC 7519, IETF, 2015.
- [8] LawScript. LawScript: AI Legal Research Platform for Pakistan. <https://www.lawscript.pk>, 2026. Accessed: 2026-04-15.
- [9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [10] Lexiber AI. Lexiber AI: Legal Research and Document Assistance for Pakistan. <https://www.lexiber.com>, 2026. Accessed: 2026-04-15.
- [11] Meta. React: The Library for Web and Native User Interfaces. <https://react.dev>, 2026. Accessed: 2026-04-12.

- [12] OpenAI. OpenAI API Reference. <https://platform.openai.com/docs/api-reference>, 2026. Accessed: 2026-04-12.
- [13] OWASP. OWASP Top Ten: The Ten Most Critical Web Application Security Risks. <https://owasp.org/www-project-top-ten/>, 2021.
- [14] Pakistan Law Bot. Pakistan Law Bot: AI-Powered Legal Assistant. <https://www.pakistanlawbot.com>, 2026. Accessed: 2026-04-15.
- [15] Peshawar High Court. Peshawar High Court: Reported Judgments – Case Management System. <https://www.peshawarhighcourt.gov.pk/PHCCMS/reportedJudgments.php>, 2026. Accessed: 2026-04-10.
- [16] PostgreSQL Global Development Group. PostgreSQL: The World’s Most Advanced Open Source Relational Database. <https://www.postgresql.org>, 2026. Accessed: 2026-04-12.
- [17] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner’s Approach*. McGraw-Hill Education, 8th edition, 2015.
- [18] Sebastián Ramírez. FastAPI: Modern, Fast Web Framework for Building APIs with Python. <https://fastapi.tiangolo.com>, 2026. Accessed: 2026-04-12.
- [19] Sindh High Court. Sindh High Court Official Portal. <https://www.sindhhighcourt.gov.pk>, 2026. Accessed: 2026-04-10.
- [20] Ian Sommerville. *Software Engineering*. Pearson Education Limited, 10th edition, 2016.
- [21] YourMunshi. YourMunshi: AI Legal Assistant for Pakistani Law. <https://www.yourmunshi.net>, 2026. Accessed: 2026-04-15.

A.4 Appendix D

A.4.1 Activity Diagram

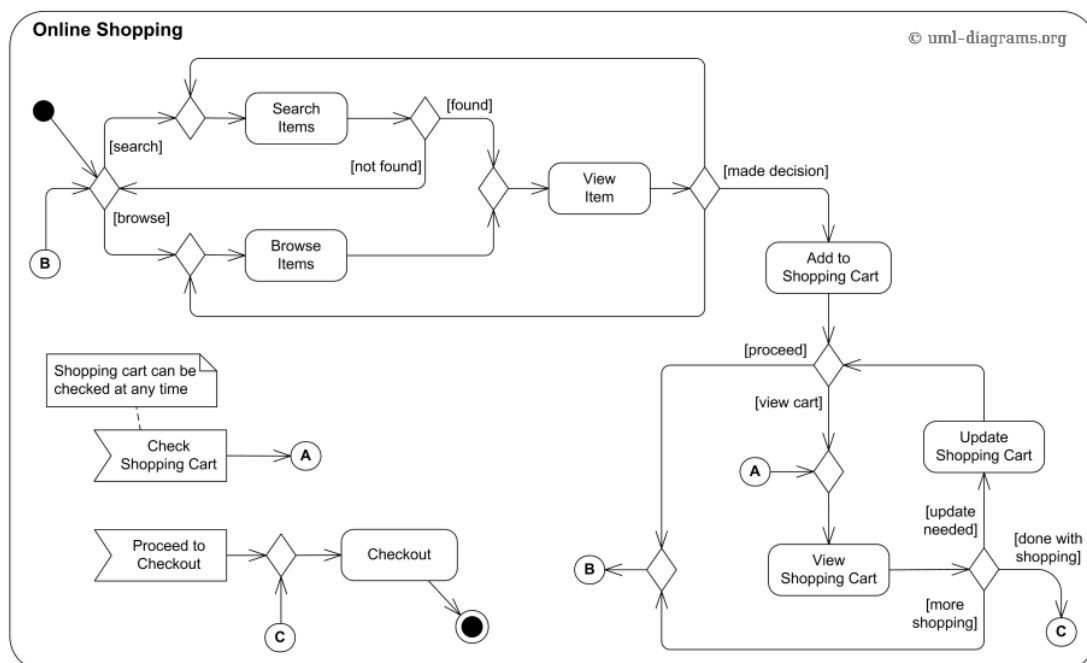


Figure A.8: Activity Diagram For Online Shopping Application